

I2C Protocol Operation

Ankit Srivastava
EE2028

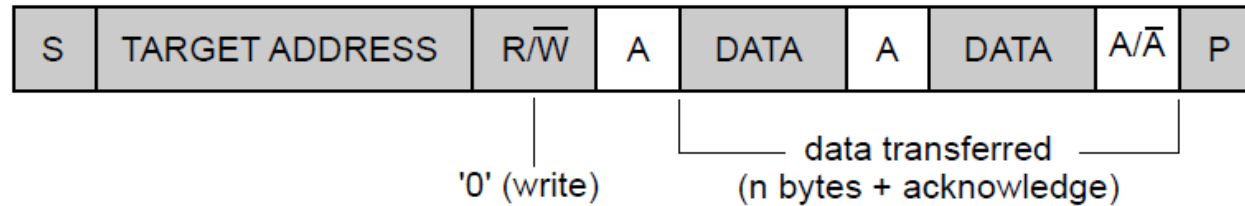
Acknowledgement: Slides adapted from Dr Rajesh Panicker



OVERVIEW

- I2C operation modes
- I2C in action
- I2C Advantages & Limitations

I2C: MASTER TRANSMITTER MODE



 from controller to target

 from target to controller

A = acknowledge (SDA LOW)

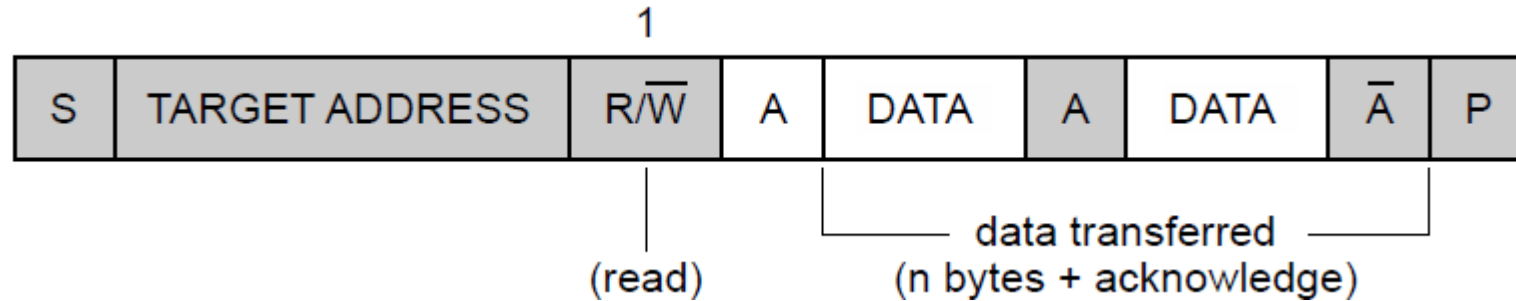
$\bar{A}$ = not acknowledge (SDA HIGH)

S = START condition

P = STOP condition

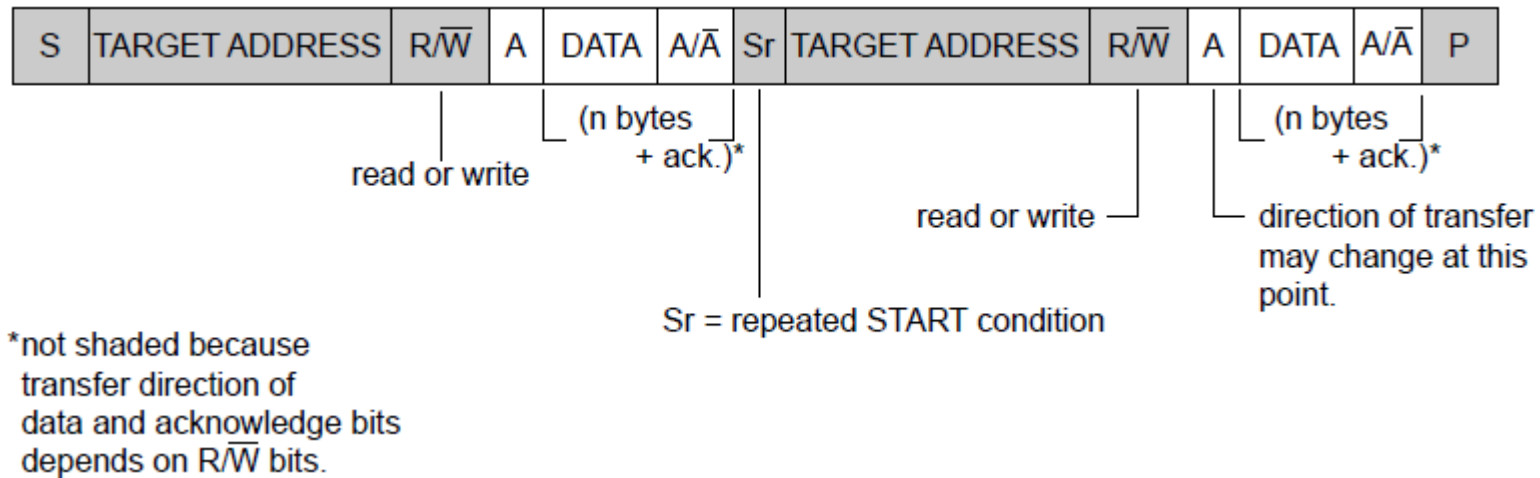
- Master transmitter transmits to slave receiver
- The slave receiver acknowledges each byte
- The master sends a P after writing the required number of bytes to terminate the transfer, or a repeated start for a change of direction to start a read transaction

I2C: MASTER RECEIVER MODE



- Master reads slave immediately after first (address) byte
- At the moment of the first acknowledge, the master transmitter becomes a master receiver, and the slave receiver becomes a slave transmitter
- This first acknowledge is still generated by the slave, subsequent acknowledges are generated by the master
- The master sends an $\bar{A}$ just prior to the P
- $\bar{A}$ from the master has only one interpretation - the master doesn't want anymore data
- It will follow the $\bar{A}$ with a P to finish the transfer, or an Sr to start a new transfer / change the direction

I2C: CHANGE OF DIRECTION



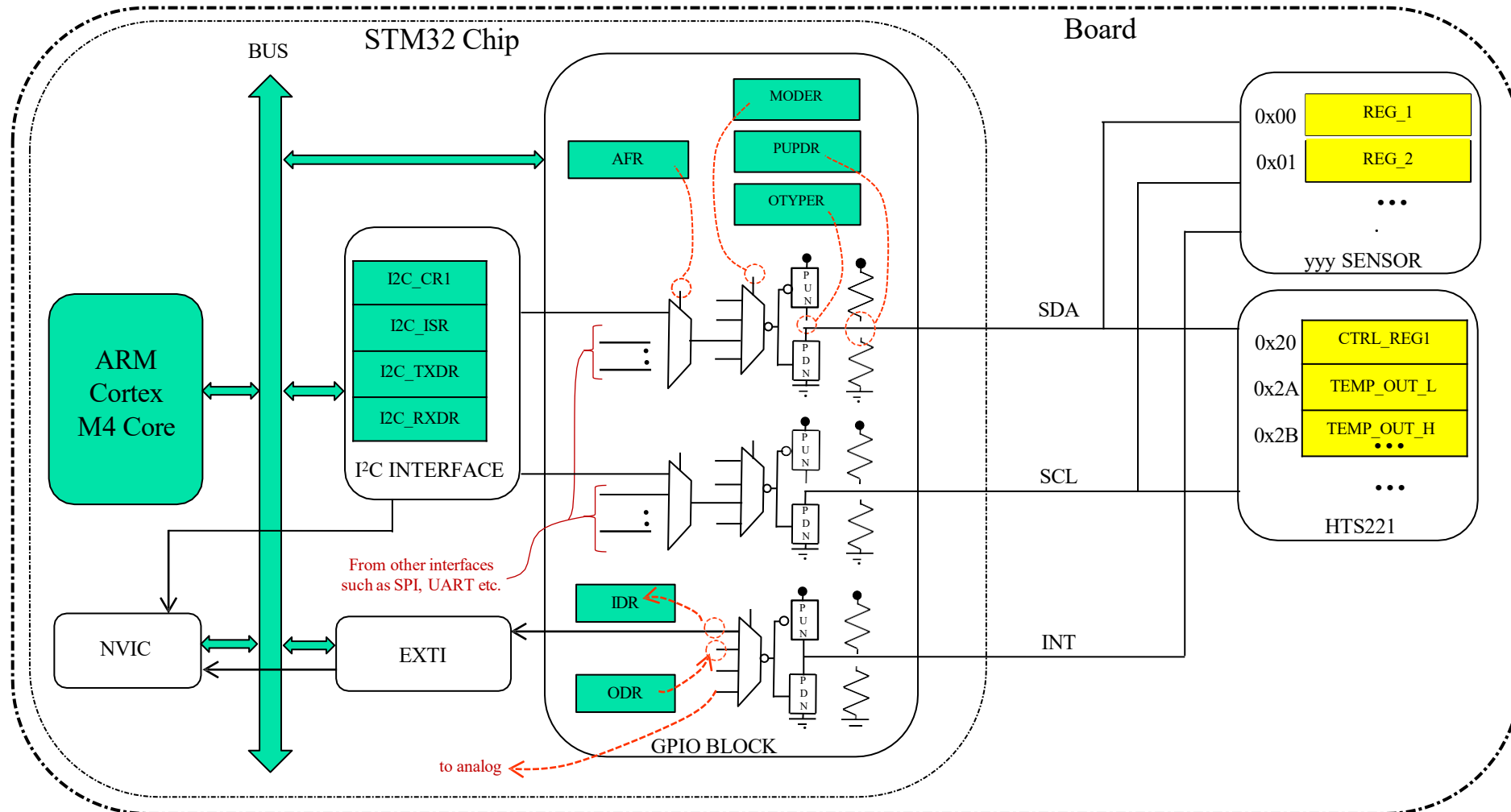
- During a change of direction within a transfer (write->read or read->write), a START condition (Sr) and slave address (with R/W bit reversed) are both repeated without a preceding P
- Essentially, it is similar to two separate transfers, but without P in between
- Sr -> sending S without a P just before
 - Allows master to retain control of the bus
- If a master-receiver sends Sr, it sends $\bar{A}$ just prior to the Sr
 - Which one is more popular : write->read or read->write?

Refer to UM10204: I2C-bus specification and user manual, Fig 13 (pg. 14)

I2C: CLOCK STRETCHING

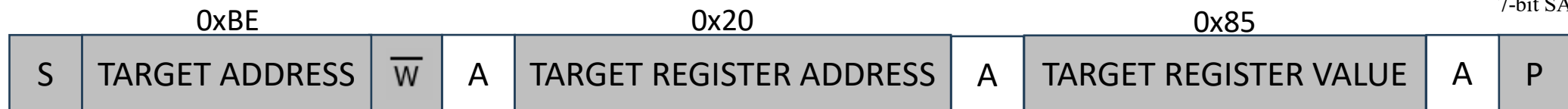
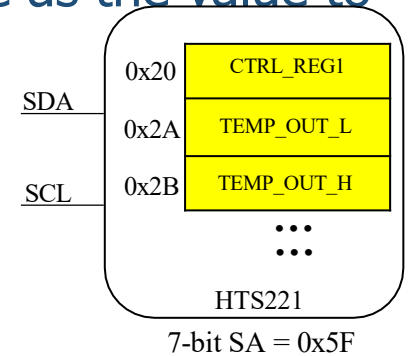
- An $\bar{A}$ from the slave has no standard meaning - it is up to the master (i.e., the software controlling the master) to interpret it and decide on the next course of action. In response to $\bar{A}$, master can generate either
 - a P to abort the transfer or
 - a Sr to start a new transfer or to retry the previous byte
- If a slave cannot receive or transmit another complete byte of data until it has performed some other function, it can hold the clock line SCL low to force the master into a wait state
 - Clock will not appear on the SCL line - in open drain, if anyone drives a low, the line will remain low
 - Data transfer then continues when the slave is ready for another byte of data and releases clock line SCL
 - This is called clock stretching - the only scenario in which the slave controls SCL
 - Not all I2C devices support clock stretching well
- What if the master does not receive the data properly from the slave? Can it not acknowledge?

I2C IN ACTION : HTS221



I2C: WRITING AN I2C SLAVE REGISTER

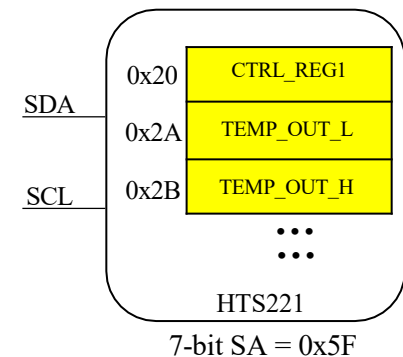
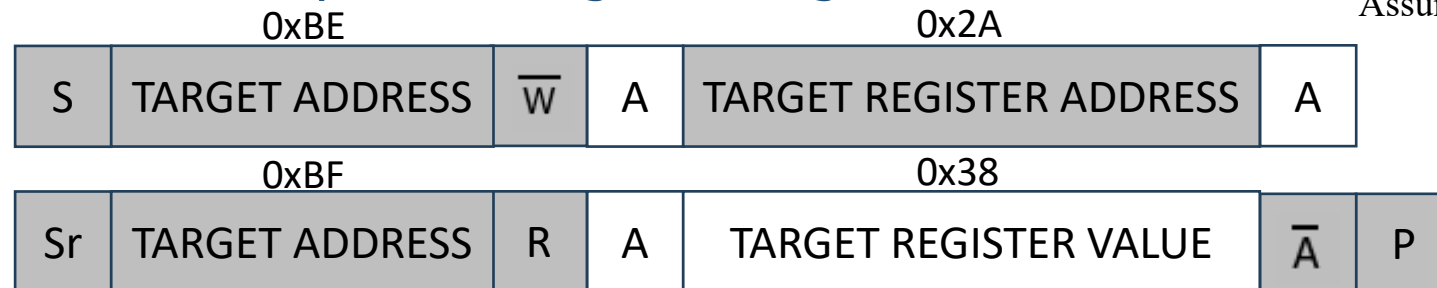
- Typically, every register inside an I2C device has 8-bit address, containing an 8-bit value
- I2C protocol defines the meaning of only the first byte sent over the I2C (SA+R/W)
- The meaning of the rest of the bytes is a matter of interpretation by the receiving party
 - A programmer needs to read the datasheet of the specific slave device, figure out how it will interpret the bytes sent, and send values appropriately to meet the needs of the application
- Typically, the second byte written by the master is interpreted by the slave as the address of the register within the slave
- Typically, the third byte written by the master is interpreted by the slave as the value to be written to the register addressed in by the previous byte



I2C: READING AN I2C SLAVE REGISTER

- Reading a register typically involves sending (writing) the address of the register to be read, followed by a read to get the register content

Example: Reading from TEMP_OUT_L register of HTS221.
Assume TEMP_OUT_L contains 0x38 (value_LSB)



- Many sensors give 16-bit readings (separate LSB and MSB registers).

- How can we read 2 registers?

0xBE is slave address (0x5F) with Read/Write set to 0 for Write
0xBF is slave address (0x5F) with Read/Write set to 1 for Read

- Write LSB address, read value_LSB, write MSB address, read value_MSB
- Alternative: Autoincrement - reading multiple registers in one transaction
 - The master reads two bytes in a single read transaction. The first byte is the content of the register whose address was written in the preceding write. The second byte is the content of the register with address++, i.e., a pointer internal to the slave keeps track of which register is read, and increments it with each byte read
 - Only if the sensor supports it. May need to be enabled/configured explicitly
 - Can be used for writing to multiple registers too, not just for reading

I2C: ADVANTAGES

- Minimal wiring – just two bidirectional lines
- Supports a fairly large number of uniquely addressable devices
- No strict baud rate
- Multi-master capability
- Allows Bit-Banging
- Speed Range Up to 3.4 Mbps
- Facilitates scalability in the system



I2C: LIMITATIONS

- High Protocol Overhead
- Lower data rates compared to alternatives
- Limited Address Space
- Address Clash
- No Plug-and-Play
- Clock Stretching Increases Latency
- Handles Short-Distance Communication Only

CONCLUSION

- I2C operation modes
 - Master Transmitter
 - Master Receiver
 - Change of Direction
- I2C in action
 - Writing an I2C slave register
 - Reading and I2C slave register
- I2C Advantages & Limitations